Введение в программирование

Компьютеры понимают только очень ограниченный набор инструкций, и чтобы заставить их что-то делать, нужно четко сформулировать задание, используя эти же инструкции. **Программа** (также «*приложение*» или «*программное обеспечение*», или «*софт*») — это набор инструкций, которые указывают компьютеру, что ему нужно делать. Физическая часть компьютера, которая выполняет эти инструкции, называется ***«железом»*** или ***аппаратной частью*** (например, процессор, материнская плата и т.д.). Данный урок является началом серии уроков по программированию на языке C++ для начинающих.

Оглавление:

1. Машинный язык
2. Язык ассемблера
3. Высокоуровневые языки программирования
4. Преимущества высокоуровневых языков программирования

Машинный язык

Процессор компьютера не способен понимать напрямую языки программирования, такие как C++, Java, Python и т.д. Очень ограниченный набор инструкций, которые изначально понимает процессор, называется **машинным кодом** (или ***«машинным языком»***). То, как эти инструкции организованы, выходит за рамки данного введения, но стоит отметить две вещи.

Во-первых, каждая команда (инструкция) состоит только из определенной последовательности (набора) цифр: `0` и `1`. Эти числа называются **битами** (сокр. от «***bi**nary digi**t***») или **двоичным кодом**.

Например, одна команда машинного кода архитектуры ×86 выглядит следующим образом:

```
10110000 01100001
```

Во-вторых, каждый набор бит переводится процессором в инструкции для выполнения определенного задания (например, *сравнить два числа* или *переместить число в определенную ячейку памяти*). Разные типы процессоров обычно имеют разные наборы инструкций, поэтому инструкции, которые будут работать на процессорах Intel (используются в персональных компьютерах), с большей долей вероятности, не будут работать на процессорах Xenon (используются в игровых приставках Xbox). Раньше, когда компьютеры только начинали массово распространяться, программисты должны были писать программы непосредственно на машинном языке, что было очень неудобно, сложно и занимало намного больше времени, чем сейчас.

Язык ассемблера

Так как программировать на машинном языке — удовольствие специфическое, то программисты изобрели язык ассемблера. В этом языке каждая команда идентифицируется коротким именем (а не набором единиц с нулями), и переменными можно управлять через их имена. Таким образом, писать/читать код стало гораздо легче. Тем не менее, процессор все равно не понимает язык ассемблера напрямую. Его также нужно переводить, с помощью ассемблера, в машинный код. **Ассемблер** — это транслятор (переводчик), который переводит код, написанный на языке ассемблера, в машинный язык. В Интернете язык ассемблера часто называют просто *«Ассемблер»*.

Преимуществом Ассемблера является его производительность (точнее скорость выполнения) и он до сих пор используется, когда это имеет решающее значение. Тем не менее, причина подобного преимущества заключается в том, что программирование на этом языке адаптируется к конкретному процессору. Программы, адаптированные под один процессор, не будут работать с другим. Кроме того, чтобы программировать на Ассемблере, по-прежнему нужно знать очень много не очень читабельных инструкций для выполнения даже простого задания.

Например, вот вышеприведенная команда, но уже на языке ассемблера:

```
mov al, 061h
```

Высокоуровневые языки программирования

Для решения проблем читабельности кода и чрезмерной сложности были разработаны высокоуровневые языки программирования. C, C++, Pascal, Java, JavaScript и Perl — это всё **языки высокого уровня**. Они позволяют писать и выполнять программы, не переживая о совместимости кода с разными архитектурами процессоров. Программы, написанные на языках высокого уровня, также должны быть переведены в машинный код перед выполнением. Есть два варианта:

- компиляция, которая выполняется компилятором;
- интерпретация, которая выполняется интерпретатором.

Компилятор — это программа, которая читает код и создает автономную (способную работать независимо от другого аппаратного или программного обеспечения) исполняемую программу, которую процессор понимает напрямую. При запуске программы весь код компилируется целиком, а затем создается исполняемый файл и уже при повторном запуске программы компиляция не выполняется.

Проще говоря, процесс компиляции выглядит следующим образом:

Интерпретатор — это программа, которая напрямую выполняет код, без его предыдущей компиляции в исполняемый файл. Интерпретаторы более гибкие, но менее эффективные, так как процесс интерпретации выполняется повторно при каждом запуске программы.

Процесс интерпретации:

Любой язык программирования может быть компилируемым или интерпретируемым, однако, такие языки, как C, C++ и Pascal — компилируются, в то время как «скриптовые» языки, такие, как Perl и JavaScript — интерпретируются. Некоторые языки программирования (например, Java) могут как компилироваться, так и интерпретироваться.

Преимущества высокоуровневых языков программирования

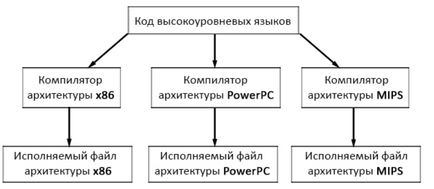
Преимущество №1: Легче писать/читать код. Вот вышеприведенная команда, но уже на языке C++:

```
a = 97;
```

Преимущество №2: Требуется меньше инструкций для выполнения определенного задания. В языке C++ вы можете сделать что-то вроде `a = b * 2 + 5;` в одной строке. В языке ассемблера вам пришлось бы использовать 5 или 6 инструкций.

Преимущество №3: Вы не должны заботиться о таких деталях, как загрузка переменных в регистры процессора. Компилятор или интерпретатор берёт это на себя.

Преимущество №4: Высокоуровневые языки программирования более портируемые под различные архитектуры (но есть один нюанс).



Нюанс заключается в том, что многие платформы, такие как Microsoft Windows, имеют свои собственные специфические функции, с помощью которых писать код намного легче. Но в таком случае приходится жертвовать портируемостью, так как функции, специфические для одной платформы, с большей долей вероятности, не будут работать на другой платформе.